Nicla Sense Env User Manual

Learn about the hardware and software features of the Arduino® Nicla Sense Env.

Author · José Bagur Last revision · 03/15/2025

This user manual provides a comprehensive overview of the Nicla Sense Env board, highlighting its hardware and software elements. With it, you will learn how to set up, configure, and use all the main features of a Nicla Sense Env board.

Hardware and Software Requirements

Hardware Requirements

- Nicla Sense Env (x1)
- Portenta C33 (x1)
- USB-C® cable (x1)

Software Requirements

- Arduino IDE 2.0+ or Arduino Web Editor
- Arduino_NiclaSenseEnv library
- Arduino Renesas Portenta Boards core (required to work with the Portenta C33 board)

> The Nicla Sense Env board is not intended as a standalone device but as a shield to work alongside a Portenta, MKR, or Nano family board. In this user manual, we will use the Portenta C33 as the main board and show how to use the Nicla Sense Env board as a shield.

Nicla Sense Env Overview

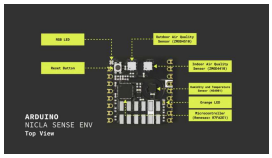
Enhance your environmental sensing capabilities with the Nicla Sense Env board. This board combines three cutting-edge sensors from Renesas® with the Arduino ecosystem's ease of integration and scalability. This board is well-suited for augmenting your Portenta or MKR-based projects with environmental sensing capabilities.

The Nicla Sense Env includes an ultra-low power temperature and humidity sensor, complemented by two sophisticated industrial-grade gas sensors capable of assessing air quality in indoor and outdoor settings. Its compact dimensions (22.86 x 22.86 mm) and sturdy build make the Nicla Sense Env an excellent choice for projects that demand sensor fusion and the computational capabilities of Arduino boards.

Nicla Sense Env Architecture Overview

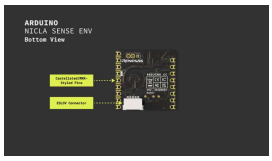
The Nicla Sense Env features a secure, certified, and durable design that suits various applications, such as industrial automation, building automation, and prototyping.

The top view of the Nicla Sense Env board is shown in the image below:



The Nicla Sense Env main components (top view)

The bottom view of the Nicla Sense Env board is shown in the image below:



The Nicla Sense Env main components (bottom view)

Here's an overview of the board's main components shown in the images above:

Microcontroller: At the heart of the Nicla Sense Env is a Renesas RA2E1 microcontroller. This entry-level single-chip microcontroller, known as one of the industry's most energy-efficient ultra-low-power microcontroller, is based on a 48 MHz Arm® Cortex®-M23 core with up to 128 KB code flash and 16 KB SRAM memory.

Onboard humidity and temperature sensor: The Nicla Sense Env features an onboard humidity and temperature sensor, the HS4001 from Renesas. This highly accurate, ultra-low power, fully calibrated relative humidity and temperature sensor features proprietary sensor-level protection, ensuring high reliability and long-term stability.

Onboard indoor air quality sensor: The Nicla Sense Env features an onboard gas sensor, the ZMOD4410 from Renesas. This sophisticated sensor was designed to detect total volatile organic compounds (TVOC)

monitor and report indoor air quality (IAQ).

Onboard outdoor air quality sensor: The Nicla Sense Env features an onboard gas sensor, the ZMOD4510 from Renesas. This sophisticated sensor was designed to monitor and report outdoor air quality (OAQ) based on nitrogen dioxide (NO_2) and ozone (O_3) measurements.

Onboard user LEDs: The Nicla Sense Env has two onboard user-programmable LEDs; one is an orange LED, and the other one is an RGB LED.

ESLOV connector: The Nicla Sense Env has an onboard ESLOV connector to extend the board communication capabilities via I^2C.

Surface mount: The castellated pins of the board allow it to be positioned as a surface-mountable module.

Board Libraries

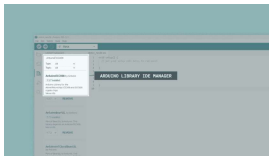
The `Arduino_NiclaSenseEnv` library contains an application programming interface (API) to read data from the board and control its parameters and behavior over I²C. This library supports the following:

- Board control (sleep, reset, and factory reset)
- Board configuration (I²C address configuration)
- Onboard RGB LED control

- Onboard indoor air quality sensor control (sulfur detection, odor intensity, ethanol level, TVOC, CO_2, IAQ measurements)
- Onboard outdoor air quality sensor control (NO_2, O_3, OAQ measurements)
- Temperature and humidity sensor control
- UART comma-separated values (CSV) output

The Portenta, MKR, Nano and UNO (R4) family boards support the `Arduino_NiclaSenseEnv` library.

To install the `Arduino_NiclaSenseEnv` library, navigate to `Tools > Manage libraries...` or click the **Library Manager** icon in the left tab of the Arduino IDE. In the Library Manager tab, search for `Arduino_NiclaSenseEnv` and install the latest version of the library.



Installing the board's library in the Arduino IDE

Pinout

The full pinout is available and

Nicla Sense Env pinout

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

Nicla Sense Env datasheet

Schematics

The complete schematics are available and downloadable as PDF from the link below:

Nicla Sense Env schematics

STEP Files

The complete STEP files are available and downloadable from the link below:

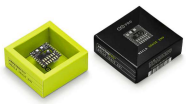
Nicla Sense Env STEP files

First Use

Unboxing the Product

Let's check out what is inside the box of the Nicla Sense Env board. Besides the board, you will find an ESLOV cable inside the box, which can connect the Nicla Sense Env with other supported Arduino boards with an onboard ESLOV connector

boards). The board's MKR-styled pins can also connect the Nicla Sense Env to other supported Arduino boards (Nano family), but 2.54 mm header pins (not included) must be soldered to the MKR-styled board pins.



Unboxing the Nicla Sense Env

The Nicla Sense Env is not a standalone device but a shield for an Arduino-supported board from the Portenta, MKR, or Nano board families. This user manual will use a Portenta C33 as the main or host board and the Nicla Sense Env as a shield or client board connected through the included ESLOV cable.

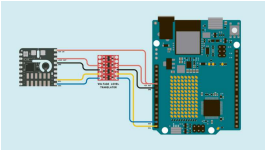
Connecting the Board

As shown in the image below, the Nicla Sense Env can be connected to a Portenta or MKR family board using the onboard ESLOV connector and the included ESLOV cable. Alternatively, you can connect the Nicla Sense Env as a shield by using the MKR-style pins on the Portenta or MKR family boards.

Connecting the Nicla Sense Env

For other compatible boards, such as those from the Nano family, the Nicla Sense Env can also be connected using the 2.54 mm pins of the Nicla Sense Env board.

Important note: The Nicla Sense Env board operates at 3.3 VDC, and while its pins are 5 VDC tolerant, we recommend using a level translator when connecting it to 5 VDC-compatible Arduino boards to ensure safe communication and prevent potential damage to the components. This connection can be made either through the Nicla Sense Env's ESLOV connector or its dedicated I2C pins (`I2C0`), as illustrated in the image below.



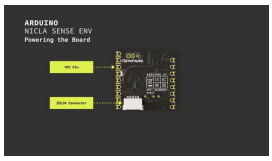
Connecting the Nicla Sense Env to a 5 VDC-compatible Arduino board

As shown in the image above, the use of a dedicated level translator between the Nicla Sense Env board and the 5 VDC-compatible Arduino board, in

Powering the Board

The Nicla Sense Env can be powered by:

- Using the onboard **ESLOV connector**, which has a dedicated +5 VDC power line regulated onboard to +3.3 VDC.
- Using an **external +3.3 VDC power supply** connected to the `VCC` pin (please refer to the **board pinout section** of the user manual).



Different ways to power the Nicla Sense Env

The Nicla Sense Env's `VCC` pin can be connected only to a +3.3 VDC power supply; any other voltage will permanently damage the board. Furthermore, the `VCC` pin does not have reverse polarity protection. Double-check your connections to avoid damaging the board.

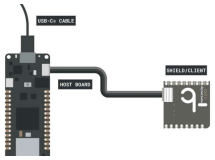
In this user manual, we will use the board's ESLOV connector to power it.

Hello World Example

`hello world` example used in the Arduino ecosystem: the `Blink`. We will use this example to verify the Nicla Sense Env's connection to the host board (a Portenta C33) via ESLOV, the host board's connection to the Arduino IDE, and that the `Arduino_NiclaSenseEnv` library and both boards, the shield and the host, are working as expected. This section will refer to the Nicla Sense Env as a shield or client.

> We are using the API of the `Arduino_NiclaSenseEnv` library with the host board (Portenta C33) to control the Nicla Sense Env (shield).

First, connect the shield to the host board via ESLOV, as shown in the image below, using an ESLOV cable (included with your Nicla Sense Env):



Connecting the Nicla Sense Env to the host board via ESLOV

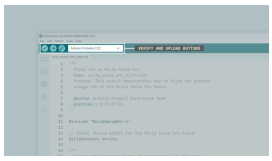
Now, connect the host board to your computer using a USB-C® cable, open the Arduino IDE, and connect the host board to it.

If you are new to the Portenta C33, please refer to the board's user manual for more detailed information.

Copy and paste the example sketch below into a new sketch in the Arduino IDE:

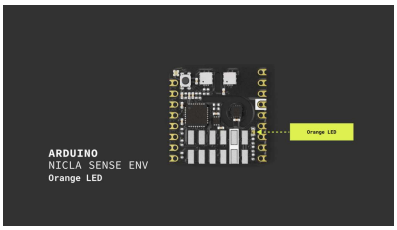
```
/**
  Blink Example on I
  Name: nicla_sense_
  Purpose: This ske
  orange LED of the

  @author Arduino P
  @version 1.0 31/0
*/

// Include the Nicla
#include "NiclaSens

// Global device ob
NiclaSenseEnv devic

/**
  Toggles the onboa
  @param led Refere
*/
void toggleLED(Oran
    // Turn on the l
    led.setBrightne
    delay(1000);

    // Turn off the
    led.setBrightne
    delay(1000);
```

To upload the sketch to the host board, click the **Verify** button to compile the sketch and check for errors, then click the **Upload** button to program the device with the sketch.



Uploading a sketch to the host board (Portenta C33) in the Arduino IDE

You should see the onboard orange LED of your Nicla Sense Env board turn on for one second, then off for one second, repeatedly.



Board Management

This section of the user manual outlines how to manage the onboard sensors and main features of the Nicla Sense Env board using the `Arduino_NiclaSenseEnv` library API. It also explains how to perform essential tasks such as retrieving the board's information, managing sensor states, resetting the board, and putting it into deep sleep mode.

Board Information

Detailed information from the

software revision, and UART communication settings, can be retrieved using the `Arduino_NiclaSenseEnv` library API. The example sketch shown below retrieves that information using a dedicated function called `printDeviceInfo()`:

```
/**
  Board Information
  Name: nicla_sense_
  Purpose: This ske

  @author Sebastián
  @version 1.0 31/0
*/

// Include the Nicl
#include "NiclaSens

// Global device ob
NiclaSenseEnv devic

/**
  Prints detailed d
  This function out
  the device I2C ad
*/
void printDeviceInf
    Serial.println(
    Serial.print("-
    Serial.print(de
    Serial.println(
    Serial.print("-
    Serial.println(
    Serial.print("-
```

Here is a detailed breakdown of the `printDeviceInfo()` function and the `Arduino_NiclaSenseEnv` library API functions used in the `printDeviceInfo()` function:

of the board. This is useful for identifying the board when multiple devices are connected to the same I^2C bus.

`serialNumber()`: Outputs the board's unique serial number. Each board's serial number is unique and can be used for tracking, inventory management, or validating its authenticity.

`productID()`: Provides the product ID, which specifies the exact model or version of the board.

`softwareRevision()`: This displays the current firmware version installed on the board. Keeping the firmware updated is critical for security, performance, and access to new features, making this information valuable for maintenance and support.

`UARTBaudRate()`: Shows the baud rate used for UART communications.

`CSVDelimiter()`: Reveals the delimiter used in CSV outputs. This detail is vital for developers who process or log data, as it affects how data is parsed and stored.

`isDebuggingEnabled()`: Indicates whether the debugging mode is active. Debugging can provide additional output that helps diagnose issues or for development purposes.

`isUARTCSVOutputEnabled()`: Shows whether CSV output through UART is enabled. This setting is

for analysis or reporting, as it impacts how data is exported from the board.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:

Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch **here**.

Onboard Sensors Management

Efficient management of the Nicla Sense Env's onboard sensors is important for optimizing its performance and power usage. The sketch shown below demonstrates how to manage (turn on or off) the onboard sensors (temperature, relative humidity, and air quality) of the Nicla Sense Env and check their status using the `Arduino_NiclaSenseEnv` library API:

```
/**
  Onboard Sensors Ma
  Name: nicla_sense_
  Purpose: This ske

  @author Arduino P
  @version 1.0 31/05
*/

// Include the Nicla
#include "NiclaSens

// Global device obj
NiclaSenseEnv devic

void setup() {
    // Initialize se
    Serial.begin(115
    for (auto startI

    if (device.begi
        // Disable a
        Serial.print
        device.tempe
        device.indoo
        device.outdo

        // Check the
```

This example sketch initializes the Nicla Sense Env board, disables all onboard sensors and then checks and prints the status of each sensor on the Arduino IDE's Serial Monitor. Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

`temperatureHumiditySensor().setEnabled(false)`: Disables the onboard temperature and humidity sensor.

indoor air quality sensor.

`outdoorAirQualitySensor().setEnabled(false)`: Deactivates the onboard outdoor air quality sensor.

`temperatureHumiditySensor().enabled()`: Checks if the onboard temperature and humidity sensor is active.

`indoorAirQualitySensor().enabled()`: Indicates whether the onboard indoor air quality sensor is currently enabled.

`outdoorAirQualitySensor().enabled()`: Confirms if the onboard outdoor air quality sensor is operational.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:

Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch here.

Board Reset

Resetting the Nicla Sense Env is important for troubleshooting and ensuring the device operates cleanly. It is handy

to the configuration or when an unexpected behavior occurs.

The example sketch below demonstrates how to reset the Nicla Sense Env using the `Arduino_NiclaSenseEnv` library API. It also shows how to verify that the board has been reset by turning off the temperature sensor before the reset and checking its status after the reset.

```
/**
  Board Reset Examp
  Name: nicla_sense_
  Purpose: This ske
  using the Arduino_
  by disabling and

  @author Arduino P
  @version 1.0 31/0
*/

// Include the Nicla
#include "NiclaSens

// Global device ob
NiclaSenseEnv devic

void setup() {
    // Initialize s
    Serial.begin(11
    for (auto startI

    if (device.begi
        // Disable
        Serial.prin
        device.temp

        // Check th
```

This example shows that the temperature sensor, disabled before the reset, is re-enabled

settings have been reset to their defaults. Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

- `device.temperatureHumiditySensor().setEnabled(false)`: Disables the onboard temperature and humidity sensor.
- `device.reset()`: This function reboots the Nicla Sense Env, clearing all temporary settings.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:

Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch here.

Low Power Mode Management

Saving energy is vital for many projects, particularly those deployed in remote areas or with a limited power supply. The Nicla Sense Env supports a deep sleep mode that can help to

Deep sleep is essential for extending battery life and minimizing energy consumption when the board is not collecting data or performing tasks. It is necessary for battery-powered or power-constrained applications.

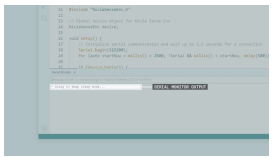
The example sketch shown below demonstrates how to put the Nicla Sense Env board into deep sleep mode using the `Arduino_NiclaSenseEnv` library API:

```
/**
  Low Power Mode Ma
  Name: nicla_sense_
  Purpose: This ske
  into deep sleep m

  @author Arduino P
  @version 1.0 31/0
*/

// Include the Nicl
#include "NiclaSens

// Global device ob
NiclaSenseEnv devic

void setup() {
    // Initialize s
    Serial.begin(11
    for (auto startl

    if (device.begi
        // Putting
        Serial.prin
        device.deep
    } else {
        Serial.prin
    }
```

Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

- `device.deepSleep()`: This function puts the Nicla Sense Env board into a deep sleep state, minimizing power consumption to the lowest possible level.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:



Example sketch output in the Arduino IDE's Serial Monitor

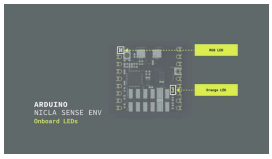
Waking up a Nicla Sense Env board from deep sleep mode can only be done by a hardware reset.

You can download the example sketch here.

LEDs

This section of the user manual explains how to control both the onboard orange and RGB and LEDs of the Nicla Sense Env board using the

library API. The LEDs can be used to provide visual feedback for various operations, such as indicating status, warnings, or sensor errors. This section covers the basic usage of both LEDs, including turning them on, changing colors, and adjusting its brightness.



The onboard LEDs of the Nicla Sense Env board

Orange LED

The onboard orange LED on the Nicla Sense Env board can be controlled using the `Arduino_NiclaSenseEnv` library. The example sketch shown below shows how to smoothly increase and decrease the brightness of the onboard orange LED. The LED pulses continuously in the `loop()` function.

```
/**
  Orange LED Contro
  Name: nicla_sense_
  Purpose: This ske
  by increasing and

  @author Arduino P
  @version 1.0 31/05
*/

// Include the Nicla
#include "Arduino_N

// Global device obj
NiclaSenseEnv device

// Initial brightnes
int brightness = 0;

// Amount to increas
int fadeAmount = 5;

void setup() {
    // Initialize se
    Serial.begin(115
    for (auto startI

    if (device.begi
```

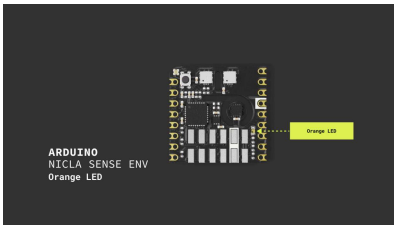
Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

- `device.begin()`: Initializes the Nicla Sense Env board, setting up communication with all onboard sensors and components, including the orange LED.
- `orangeLED.setBrightness(uint8_t brightness)`: Adjusts the brightness of the orange LED. The brightness ranges from `0` (off) to `255` (full

increases from `0` to `255` and then decreases back to 0, creating a smooth pulsing effect.

`fadeAmount`: Controls the rate of change of brightness. When the brightness reaches either 0 or 255, the direction of change is reversed, making the LED brightness smoothly cycle up and down.

After uploading the example sketch to the Nicla Sense Env board, you should see the orange LED smoothly increase and decrease in brightness, creating a continuous pulsing effect.



You can download the example sketch **here**.

RGB LED

The onboard RGB LED on the Nicla Sense Env board can be controlled using the `Arduino_NiclaSenseEnv` library. The example sketch shown below shows how to turn on the LED with different colors and then turn it off using the `setColor()` and `setBrightness()` functions. The LED colors cycle continuously in the `loop()` function

```
/**
  RGB LED Control E
  Name: nicla_sense_
  Purpose: This ske
  different colors a

  @author Arduino P
  @version 1.0 31/05
*/

// Include the Nicla
#include "Arduino_N

// Global device obj
NiclaSenseEnv device

void setup() {
    // Initialize se
    Serial.begin(115
    for (auto startI

    if (device.begi
        Serial.print
    } else {
        Serial.print
    }
}
```

Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

- `device.begin()`: Initializes the Nicla Sense Env board, setting up communication with all onboard sensors and components, including the RGB LED.
- `rgbLED.setColor(uint8_t red, uint8_t green, uint8_t blue)`: This function sets the RGB LED to a specific color

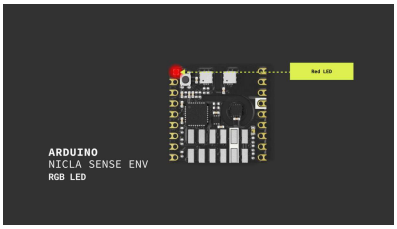
components. Each value can range from `0` (off) to `255` (full brightness). In the example, the RGB LED cycles through red (255, 0, 0), green (0, 255, 0), and blue (0, 0, 255).

`rgbLED.setBrightness( uint8_t brightness)` : Adjusts the brightness of the RGB LED. The value ranges from `0` (off) to `255` (full brightness). In the sketch, the brightness is set to 255 (maximum) when the LED is on, and to 0 (off) when the LED is turned off.

After uploading the example sketch to the Nicla Sense Env board, you should see the following output in the Arduino IDE's Serial Monitor:

Example sketch output in the Arduino IDE's Serial Monitor

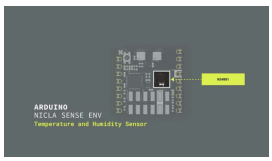
You should also see the onboard RGB LED of your Nicla Sense Env board turn on red for one second, then green for one second, then blue for one second, and finally turn off, repeating this cycle.



You can download the example sketch **here**.

Temperature and Humidity Sensor

The Nicla Sense Env board has an onboard temperature and humidity sensor, the HS4001 from Renesas. The HS4001 is a highly accurate, ultra-low power, fully calibrated automotive-grade relative humidity and temperature sensor. Its high accuracy, fast measurement response time, and long-term stability make the HS4001 sensor ideal for many applications ranging from portable devices to products designed for harsh environments.



The HS4001 sensor of the Nicla Sense Env board

The example sketch below demonstrates how to read temperature and humidity data from the HS4001 sensor using the `Arduino_NiclaSenseEnv` library API. The sketch will report the temperature and humidity values to the Arduino IDE's Serial Monitor every 2.5 seconds.

```
/**
  Temperature and Hu
  Name: nicla_sense_
  Purpose: This ske
  the HS4001 sensor

  @author Arduino P
  @version 1.0 31/05
*/

// Include the Nicla
#include "NiclaSens

// Global device obj
NiclaSenseEnv devic

/**
  Displays temperatu
  @param sensor Refe
*/
void displaySensorDa
    if (sensor.enabl
        float tempe
        if (isnan(te
            Serial.p
        } else {
            Serial.p
            Serial.p
```

Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

- `temperatureHumiditySensor()`: Retrieves the temperature and humidity sensor object from the Nicla Sense Env.
- `sensor.enabled()`: Checks if the sensor is currently enabled.
- `sensor.temperature()`: Reads the current temperature value from the sensor. The reading is

`sensor.humidity()`: Reads the current relative humidity value from the sensor.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:

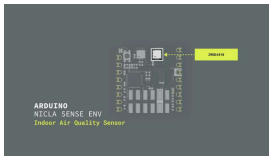
Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch **here**.

Indoor Air Quality Sensor

The Nicla Sense Env board features an onboard air quality sensor, the ZMOD4410 from Renesas. The ZMOD4410 is a highly integrated digital gas sensor module for indoor air quality monitoring. It provides measurements of total volatile organic compounds (TVOCs), estimates CO_2 levels, and monitors indoor air quality (IAQ), making it suitable for applications such as smart home devices, air purifiers, and HVAC systems. Its compact size, low power consumption, and high sensitivity make this sensor

air quality monitoring applications.



The ZMOD4410 sensor of the Nicla Sense Env board

> Every ZMOD sensor is electrically and chemically calibrated during Renesas production. The calibration data is stored in the non-volatile memory (NVM) of the sensor module and is used by the firmware routines during sensor initialization. ZMOD sensors are qualified for a 10-year lifetime (following the JEDEC JESD47 standard) without the need for recalibration.

The example sketch below demonstrates how to read air quality data from the ZMOD4410 sensor using the `Arduino_NiclaSenseEnv` library API. The sketch reports indoor air quality values to the Arduino IDE's Serial Monitor every 5 seconds.

Important: The ZMOD4410 supports several operation modes, each with specific sample rates and warm-up requirements. For IAQ measurements, the sensor can take a sample every three

up samples, meaning a total warm-up time of 3 minutes. In ultra-low-power mode, the sensor can take samples every 90 seconds but requires only 10 warm-up samples, meaning it takes 15 minutes to fully warm-up.

```
/**
  Indoor Air Quality
  Name: nicla_sense_
  Purpose: This ske
  ZMOD4410 sensor ou

  @author Arduino P
  @version 1.0 31/05
*/

// Include the Nicla
#include "NiclaSens

// Global device obj
NiclaSenseEnv devic

/**
  Displays air qual
  @param sensor Refe
*/
void displaySensorDa
    if (sensor.enabl
        Serial.print
        Serial.print
        Serial.print
        Serial.print
        Serial.print
        Serial.print
```

Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

`indoorAirQualitySenso r()`

sensor object from the Nicla Sense Env.

`sensor.enabled()`: Checks if the sensor is currently enabled.

`sensor.airQuality()`: Reads the current air quality value from the sensor.

`sensor.CO2()`: Reads the estimated CO_2 concentration from the sensor.

`sensor.TVOC()`: Reads the sensor's total concentration of volatile organic compounds.

`sensor.ethanol()`: Reads the ethanol concentration from the sensor.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:

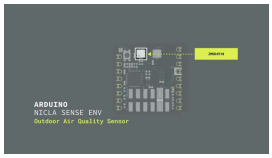
Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch here.

Outdoor Air Quality Sensor

The Nicla Sense Env board features an onboard outdoor air

a digital gas sensor module for outdoor air quality monitoring. It measures nitrogen dioxide (NO_2) and ozone (O_3) levels. It calculates an outdoor air quality index (AQI), making it suitable for air quality monitoring stations, wearable devices, and smart city infrastructure applications. Its robustness, low power consumption, and high sensitivity make this sensor an excellent choice for outdoor air quality monitoring applications.



The ZMOD4510 sensor of the Nicla Sense Env board

Every ZMOD sensor is electrically and chemically calibrated during Renesas production. The calibration data is stored in the non-volatile memory (NVM) of the sensor module and is used by the firmware routines during sensor initialization. ZMOD sensors are qualified for a 10-year lifetime (following the JEDEC JESD47 standard) without the need for recalibration.

The example sketch below demonstrates how to read air quality data from the ZMOD4510 sensor using the

outdoor air quality values to the Arduino IDE's Serial Monitor every 5 seconds.

Important: The ZMOD4510 supports several operation modes, each with specific sample rates and warm-up requirements. For NO_2/O_3 measurements, the sensor can take a sample every 6 seconds but requires 50 warm-up samples, meaning a total warm-up time of 5 minutes. In ultra-low-power O_3 mode, the sensor can take samples every two seconds, but it requires 900 warm-up samples before it is fully operational, meaning it takes 30 minutes to warm-up completely.

```
/**
  Outdoor Air Quali
  Name: nicla_sense_
  Purpose: This ske
  ZMOD4510 sensor ou

  @author Arduino P
  @version 1.0 31/05
*/

// Include the Nicla
#include "NiclaSens

// Global device obj
NiclaSenseEnv devic

/**
  Displays air qual
  @param sensor Ref
*/
void displaySensorDa
    if (sensor.enab
        Serial.prin
        Serial.prin
        Serial.prin
        Serial.prin
        Serial.prin
        Serial.prin
```

Here is a detailed breakdown of the example sketch shown before and the `Arduino_NiclaSenseEnv` library API functions used in the sketch:

- `outdoorAirQualitySensor()`: Retrieves the outdoor air quality sensor object from the Nicla Sense Env.
- `sensor.enabled()`: Checks if the sensor is currently enabled.
- `sensor.airQualityIndex()`: Reads the current outdoor air quality index value.

concentration from the sensor.

`sensor.O3()`: Reads the ozone concentration from the sensor.

After uploading the example sketch to the host board, you should see the following output in the Arduino IDE's Serial Monitor:

Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch **here**.

Communication

The Nicla Sense Env board features a UART interface for data logging purposes. This allows the board to output sensor data in `CSV` format over UART, enabling easy data logging and monitoring in various applications. The UART interface is handy for scenarios where the board needs to communicate with other microcontrollers or systems without relying on USB connections.

The example sketch below demonstrates how to read data from the UART port on the Nicla

powered through the board's `VCC` pin or the ESLOV connector of a host board and connected to another board through the UART pins. The example also requires enabling the UART output on the Nicla Sense Env.

```
/**
  UART Communicati
  Name: nicla_sense
  Purpose: This ske
  and process the d

  @author Sebastián
  @version 1.0 31/0
*/

// Include necessar
#include <map>
#include <vector>
#include <tuple>

// Define the defau
constexpr char DEFA

// Define a mapping
std::map<u_int8_t,
    {0, {"HS4001 sa
    {1, {"HS4001 te
    {2, {"HS4001 hu
    {3, {"ZMOD4510
    {4, {"ZMOD4510
    {5, {"ZMOD4510
    {6, {"ZMOD4510
    {7, {"ZMOD4510
```

Here is a detailed breakdown of the example sketch shown before and the main functionalities of the UART communication on the Nicla Sense Env:

First, the Nicla Sense Env needs to have its UART

the board over I²C to a host board and running `device.begin()` and `device.setUARTCSVOutputEnabled(true, true)`.

`Serial1.begin(38400, SERIAL_8N1)`: Initializes the `Serial1` port for UART communication with the specified baud rate and configuration.

`processCSVLine()`: A function that processes a CSV line, maps the fields to their corresponding names and stores them in a map for easy access.

The `loop()` function reads data from the UART port, processes the CSV data, and prints the parsed values to the Serial Monitor.

You can download the example sketch **here**.

Support

If you encounter any issues or have questions while working with your Nicla Sense Env board, we provide various support resources to help you find answers and solutions.

Help Center

Explore our Help Center, which offers a comprehensive collection of articles and guides for Nicla family boards. The Help Center is designed to provide in-depth technical

Nicla family help center page

Forum

Join our community forum to connect with other Nicla family board users, share your experiences, and ask questions. The Forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Nicla Sense Env.

Nicla Sense Env category in the Arduino Forum

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Nicla family boards.

Contact us page

wrong, you can edit this page here.

Was this article helpful?